



Exam Project — Django 2025–2026

Tunisian Hope and Future for Children and Youth

“Build technology that protects and empowers the next generation.”

— Python Web Programming (Django)

Contents

1	Introduction	3
2	Theme, Vision, and Scope	3
3	Problem Statement	4
4	Minimum Scope and Mandatory Deliverables	5
5	Objectives	6
6	Conceptual Framework for the Solution	6
6.1	Workflow modeling	6
6.2	Risk and intervention logic	7
6.3	Safety posture	7
7	Core Architecture and Roles	7
8	Scenarios to Demonstrate	8
8.1	Scenario 1 — Education early warning	8
8.2	Scenario 2 — Health/mental-health follow-up	9
8.3	Scenario 3 (optional) — Youth digital behavior support	9
9	Challenges and Considerations	9
10	Evaluation Metrics	10
11	Graphical User Interface and Monitoring	10
12	Advanced Data Acquisition and Web Scraping (Optional)	11
13	Technical Stack	12
13.1	Core requirements	12
13.2	Recommended packages	13
13.3	What your code must demonstrate	13
14	Advanced Tracks Inspired by Course Topics	13
15	Advanced AI, LLM, Fine-Tuning, and RAG Extensions	15
16	Governance, Risk Register, and Ethical Safeguards	18
17	Assessment Rubric	18

18 Implementation Plan	19
19 Submission Package	21
20 Scheduling, Submission, and Individual Work	21
21 Useful Links	22
22 Student Quick Checklist	23
23 Final Note	24

1 Introduction

This exam project asks you to design and implement a Django-based system that addresses a meaningful Tunisian social challenge linked to children and youth behavior. The purpose is practical and responsible: your application should help **study**, **understand**, and **improve outcomes** in sectors such as:

- education and school life,
- health and preventive care,
- mental health and psychosocial support,
- digital and social behavior of youth,
- community and family support pathways.

Your target is not to build “just another CRUD app,” but a coherent web platform with:

- a clear societal problem framing,
- a robust backend architecture,
- explicit data and safety controls,
- measurable indicators of impact and reliability.

Why this is a software-engineering exam. The core challenge is to formalize a sensitive real-world process into a rigorous digital workflow: model states and transitions, enforce validation and permissions, prevent unsafe actions, and produce evidence of correctness. This is exactly the engineering mindset expected in modern Django systems.

2 Theme, Vision, and Scope

Tunisian hope and future

Your project must contribute to a constructive vision: **early detection, prevention, support, and informed decision-making** for children and young people in Tunisia.

Stakeholders and value

Your platform should help one or more stakeholders (schools, counselors, health teams, NGOs, youth centers, researchers, local authorities, or policy actors) make better decisions using structured data and transparent workflows.

Examples of acceptable project directions

- School engagement and absenteeism early-warning dashboard.
- Student stress and well-being tracking platform with referral workflow.
- Youth digital behavior risk-monitoring and awareness portal.
- Community health follow-up for adolescents (appointments, reminders, trends).
- Multi-actor case coordination tool (teacher–family–counselor workflow).

3 Problem Statement

You must build a Django web platform that solves a clearly defined real-world use case centered on youth/children behavior and outcomes.

What “done” means for this exam

At the end of the project, a reviewer must be able to:

- run your system from the README with reproducible commands,
- execute two complete scenarios from intake to decision support output,
- observe at least one controlled failure case per scenario and safe recovery,
- verify that permissions, validation, and audit traces are enforced.

Task clarity: each task must contain

Your statement must explicitly define:

- **Population:** who is studied (age range, context).
- **Problem:** what behavior/outcome is targeted.
- **Decision maker:** who uses the output and why.
- **Operational workflow:** what users do step by step.
- **Expected value:** what gets better (faster intervention, better follow-up, fewer dropouts, etc.).
- **Validation rule:** how the platform determines success/failure for each workflow.

Data and ethics boundary

- Use **synthetic or openly licensed data** only.
- Do not include real personal, medical, or confidential records.
- Document anonymization, consent assumptions, and access-control policy.
- Explain limitations to avoid over-interpretation of behavioral indicators.

4 Minimum Scope and Mandatory Deliverables

To avoid drift, treat the following as **non-negotiable**:

- One complete Django platform with role-based workflows (minimum 3 roles).
- At least **2 operational scenarios** implemented end-to-end.
- At least **1 failure-injection case per scenario** (validation failure, timeout, unauthorized access attempt, bad file schema, etc.).
- A web dashboard/UI for operational monitoring and decision support.
- Automated tests (unit + integration) and reproducible evidence.

Clear submission logic

- If a feature is implemented but not demonstrated in evidence (screenshots/logs/tests), it is considered missing.
- If a metric is claimed but not computed from reproducible data, it is considered unsupported.
- If an AI output is shown without source/citation trace, it is considered non-auditable.

5 Objectives

The main objectives are:

- Build a coherent full-stack Django solution for a social-impact use case.
- Enforce strict data contracts (inputs/outputs/validation rules).
- Design safe authorization and privacy-aware data handling.
- Integrate measurable indicators (operational and social-impact proxies).
- Demonstrate reliability with testing, logs, and failure handling.
- Produce clear technical and decision-oriented evidence.

6 Conceptual Framework for the Solution

6.1 Workflow modeling

Your platform should be modeled as a sequence of state transitions:

- intake and validation,
- assessment/scoring or rule-based classification,
- intervention planning,
- follow-up and re-evaluation.

Each transition must be explicit and auditable.

6.2 Risk and intervention logic

You may use rule-based scoring or explainable model-assisted indicators, but you must:

- justify indicator selection,
- prevent opaque “black-box only” decisions,
- provide human-readable explanations for alerts/recommendations.

6.3 Safety posture

Define what your platform protects against:

- unauthorized data access,
- incorrect data ingestion,
- accidental disclosure of sensitive attributes,
- silent workflow failures without alerts.

7 Core Architecture and Roles

Suggested logical architecture

1. **Data Intake Layer:** forms/upload/API with schema checks.
2. **Core Domain Layer:** case records, events, intervention plans.
3. **Decision Layer:** rule engine and/or explainable scoring.
4. **Operations Layer:** dashboards, alerts, reporting, exports.
5. **Governance Layer:** auth, audit logs, privacy controls.

User roles (minimum)

- **Operator:** enters and updates records.
- **Supervisor/Counselor:** validates and plans interventions.
- **Admin/Manager:** configures policies and monitors outcomes.

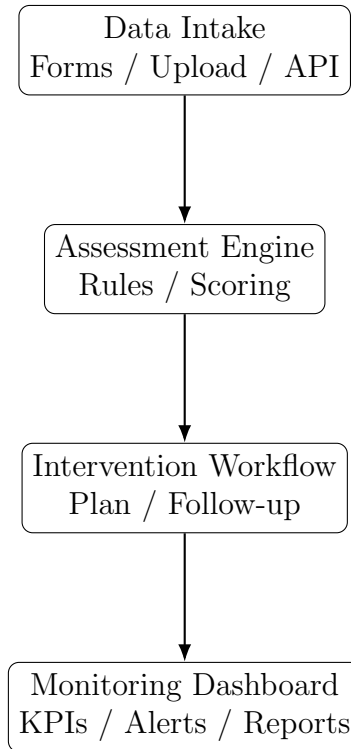


Figure 1: Reference workflow for a youth-support Django platform.

8 Scenarios to Demonstrate

8.1 Scenario 1 — Education early warning

Context. Synthetic school dataset (attendance, grades trend, behavior notes, support history).

Goal. Detect disengagement risk and produce a prioritized intervention list.

Evidence required. Valid ingestion, dashboard outputs, one failure-injection case (e.g., malformed dataset), and test results.

Minimum acceptance criteria.

- At least one risk-threshold rule is configurable by supervisor role.
- At least one intervention recommendation includes an explanation field.

- At least one export/report is generated from filtered dashboard data.

8.2 Scenario 2 — Health/mental-health follow-up

Context. Synthetic follow-up records (appointments, symptom indicators, missed sessions).

Goal. Track adherence and trigger referral/reminder workflow.

Evidence required. Workflow completion trace, alert quality indicators, one failure-injection case, and test results.

Minimum acceptance criteria.

- Missed follow-up detection must trigger a logged reminder/referral action.
- Case timeline must show who acted, when, and what changed.
- Unauthorized role action must be blocked and logged with reason.

8.3 Scenario 3 (optional) — Youth digital behavior support

Optional scenario for awareness and preventive actions linked to digital habits and online risks, with clear non-punitive intervention logic.

9 Challenges and Considerations

- Data quality and missing values in sensitive social contexts.
- False positives/false negatives in risk indicators.
- Privacy and role-based visibility constraints.
- Usability for non-technical stakeholders.
- Responsible communication of alerts (no stigmatizing labels).

10 Evaluation Metrics

Your evaluation must be measurable and include at least:

- **Workflow completion rate:** share of cases processed end-to-end.
- **Data validation pass rate:** accepted vs rejected inputs.
- **Alert precision proxy:** relevance assessment on test scenarios.
- **Recovery effectiveness:** safe behavior under injected failures.
- **Security checks coverage:** role/permission tests passed.
- **UI usability evidence:** clarity of dashboard and intervention steps.
- **Reproducibility score:** ability to run setup/tests from README.

11 Graphical User Interface and Monitoring

Your platform must provide a clear decision-oriented GUI. You may implement:

- a Django web interface (templates and/or SPA), or
- a **Flutter** client connected to Django APIs, or
- a **Kivy** desktop/mobile client connected to Django APIs.

Whichever option you choose, the same monitoring and evidence requirements below remain mandatory.

Minimum GUI scope

- Case intake and validation interface.
- Case list with filters (status, risk level, region/school/center).
- Detail view with timeline of actions.
- Dashboard with key indicators and alert summaries.
- Export/report view (CSV/PDF or equivalent structured output).

Monitoring outputs

For every key action, log at minimum:

- user role and timestamp,
- affected case id,
- action result (success/failure),
- validation or policy reason when blocked.

GUI clarity requirements

- Use explicit labels (avoid ambiguous terms such as “score” without definition).
- Show status progression visually (new, in-review, intervention, follow-up, closed).
- Separate facts from recommendations in the interface.
- Display source/evidence links for every generated alert or AI-assisted suggestion.

12 Advanced Data Acquisition and Web Scrapping (Optional)

If your project needs external public information (policies, youth programs, local service directories, educational resources, awareness content), you may implement advanced web scraping responsibly.

Hard constraints (mandatory)

- Scrape only publicly accessible and legally reusable sources.
- Respect terms of use, robots policies, and rate limits.
- Never collect personal/confidential data or bypass access controls.
- Store source URL, access date, and extraction method for every record.

Advanced scraping techniques you may demonstrate

- **Static extraction pipeline:** robust CSS/XPath selectors, schema validation, and fallback selectors.
- **Dynamic rendering extraction:** headless-browser workflow for JavaScript-rendered pages.
- **Incremental crawling:** fetch only changed/new pages using fingerprints and content hashes.
- **Deduplication and canonicalization:** normalized URL strategy and near-duplicate detection.
- **Polite concurrency:** bounded parallel requests with retry/backoff and host-level throttling.
- **Change monitoring:** alert when source structure breaks extraction rules.

Scraping quality evidence (if used)

- Source registry table (domain, license note, robots/rate-limit note, purpose).
- Extraction quality report (valid rows, failed rows, retry count, dedup ratio).
- Reproducible ingestion command and one failure injection (HTML structure change).

13 Technical Stack

13.1 Core requirements

- Python 3.11+ and Django (stable branch).
- Relational database (SQLite for prototype; PostgreSQL recommended for advanced teams).
- Django templates or API + frontend of your choice.
- Strong validation (forms/serializers/model constraints).

13.2 Recommended packages

- **API:** ‘djangorestframework’
- **GraphQL (optional):** ‘graphene-django’ or ‘strawberry-graphql’
- **gRPC (optional):** ‘grpcio’ and ‘grpcio-tools’
- **Validation:** ‘pydantic’ or ‘jsonschema’ (optional but recommended)
- **Testing:** ‘pytest’, ‘pytest-django’
- **Data handling:** ‘pandas’
- **Logging:** structured logging package or Python logging config
- **Optional UI/Charts:** ‘plotly’, ‘chart.js’, or equivalent

13.3 What your code must demonstrate

- Reliable role-based access checks.
- Input validation and failure-safe behavior.
- Scenario-based tests and reproducible commands.
- Decision traces understandable by non-technical reviewers.
- If GraphQL/gRPC are used: strict schema/contracts, typed errors, and access-control checks per operation.

14 Advanced Tracks Inspired by Course Topics

To align this exam with the full Django subject catalogue, each team must select **at least 2 advanced tracks** below and provide concrete evidence in code/tests/docs.

Track A — Architecture and performance

- Modular design (service layer or bounded-context style) and clear domain boundaries.
- ORM optimization evidence (query plans, N+1 removal, indexing rationale).
- Optional async endpoint or worker integration with concurrency-safe behavior.

Track B — Security and privacy by design

- Threat model summary (assets, abuse cases, mitigations).
- Strong authentication/session practices and role/permission policy checks.
- Sensitive-field handling (redaction/anonymization) and auditability.

Track C — API and integration quality

- Contracted REST API (clear request/response schemas and validation errors).
- Optional GraphQL endpoint with explicit schema design and resolver authorization.
- Optional gRPC service for internal high-reliability flows with ‘.proto’ contracts.
- Integration reliability (timeouts, retries, idempotent behavior where needed).
- Optional event-driven or webhook workflow with traceability.

Track D — Observability and reliability

- Structured logs with correlation ids or equivalent trace fields.
- Health and error indicators (dashboard, report, or service endpoint).
- Failure injection evidence and safe recovery behavior.

Track E — Responsible AI/decision-support (optional but valued)

- Explainable scoring/rule support (no opaque “magic” recommendation).
- Human override path for sensitive decisions.
- Clear limits and bias-risk discussion in documentation.

Track F — Advanced AI/LLM and RAG (optional excellence track)

- Integrate an LLM-assisted component through a safe service boundary (never direct unrestricted tool execution).
- Use **advanced RAG** design beyond naive top-k retrieval.
- Add measurable evaluation (faithfulness, citation coverage, hallucination rate proxy, latency/cost).
- Enforce safety controls (prompt-injection handling, output validation, policy filters, human override for high-risk actions).

15 Advanced AI, LLM, Fine-Tuning, and RAG Extensions

This section is optional, but teams who implement it rigorously can demonstrate higher technical maturity. If you use AI components, keep them auditable and reproducible.

Protocol requirements: MCP, A2A, ACP, and ANP

When AI-agent features are implemented, teams should explicitly define and document these protocol choices:

- **MCP (Model Context Protocol):** standardize how tools, resources, and context are exposed to LLM/agent components.
- **A2A (Agent-to-Agent):** define inter-agent message contracts, hand-off rules, and failure behavior between planner/executor/reviewer roles.

- **ACP (Agent Communication Protocol):** enforce structured envelopes (intent, input schema, output schema, confidence, trace id) for reliable exchanges.
- **ANP (Agent Network Protocol):** define network-level routing/governance for multi-agent execution (timeouts, retries, isolation boundaries, and observability hooks).

You may adapt naming to your chosen framework, but your architecture must clearly show equivalent responsibilities.

A) Modern LLM integration patterns in Django

- **Gateway pattern:** keep LLM calls behind a Django service layer with request/response schemas.
- **Structured outputs:** require JSON schema-constrained outputs for downstream reliability.
- **Tool-calling discipline:** explicit allow-lists, argument validation, and full audit logs.
- **Fallback design:** deterministic rule-based fallback when model confidence is low or policies fail.

B) Advanced RAG techniques you may use

- **Hybrid retrieval:** combine lexical (BM25) and vector retrieval for robustness.
- **Reranking stage:** apply a cross-encoder or reranker after initial retrieval.
- **Query rewriting/decomposition:** split complex questions into smaller retrieval sub-queries.
- **Metadata-aware retrieval:** filter by age group, region, source type, or date for relevance and safety.
- **Multi-hop retrieval:** chain evidence across multiple documents before generating output.
- **Graph-RAG option:** represent entities/relations (school, counselor, case event) for relation-aware retrieval.

- **Context compression:** keep only evidence-relevant chunks to control token cost and noise.

C) Fine-tuning and adaptation options

- **Prompt tuning first:** start with prompt/system-instruction optimization before model fine-tuning.
- **PEFT fine-tuning:** LoRA/QLoRA-style adaptation when data and compute allow.
- **Task adapters:** separate adapters per task (triage, summarization, recommendation explanation).
- **Data curation protocol:** use synthetic or anonymized records with quality and leakage checks.

D) AI evaluation and reliability requirements

- Track **faithfulness:** is every claim grounded in retrieved evidence?
- Track **citation quality:** are cited snippets relevant and sufficient?
- Track **safety:** refusal/abstention behavior on unsafe or under-specified requests.
- Track **cost and latency:** p50/p95 latency and token/compute budget.
- Track **protocol reliability:** MCP/A2A/ACP/ANP contract violations, timeout rate, and handoff success rate.
- Include at least one **regression test set** for prompt/model changes.

E) Required safeguards when AI is used

- No autonomous high-impact decision without human validation.
- Explicit disclaimer in UI: decision support, not clinical/legal final authority.
- Record model/version, prompt template version, retrieval config, and timestamp per run.
- Block unsupported instructions and sanitize tool inputs/outputs.

F) Integration with web-scraped knowledge (if used)

- Scraped content must pass schema and quality checks before indexing for RAG.
- Index entries must keep provenance (source URL, extraction date, chunk id).
- Retrieval must prioritize trusted and validated sources.
- High-impact suggestions must include at least one citation from validated sources.

16 Governance, Risk Register, and Ethical Safeguards

Include a short risk register in `docs/` with at least:

- **Data risks:** leakage, poor quality, missing context.
- **Decision risks:** false alarms, missed-risk cases, unfair treatment.
- **Operational risks:** service outage, unauthorized actions, silent failures.
- **Mitigations:** technical controls, human review steps, escalation logic.

17 Assessment Rubric

Evaluation should remain evidence-driven and balanced:

- **Problem framing and social relevance:** 20%
- **Solution architecture and implementation quality:** 25%
- **Security, privacy, and governance discipline:** 20%
- **Validation quality (tests, failure handling, reproducibility):** 20%
- **Communication quality (slides/demo/report clarity):** 15%

18 Implementation Plan

Week 1 — Scope, roles, and contracts

- Freeze the problem statement in one paragraph (population, problem, decision maker, expected value).
- Define roles and permissions matrix (who can create, validate, escalate, close).
- Define workflow states and transition rules (including blocked transitions).
- Define input/output schemas and validation contracts.
- Prepare synthetic/open data, ethics notes, and data dictionary.
- Select at least 2 advanced tracks and write acceptance criteria for each.

Week 1 deliverables:

- docs/problem_statement.md
- docs/roles_matrix.md
- docs/state_machine.md
- initial dataset sample and data dictionary
- initial API/UI wireframe sketch

Week 2 — Core platform and Scenario 1

- Build Django models, authentication, authorization, and core workflow actions.
- Implement Scenario 1 from intake to dashboard output.
- Implement validation errors and user-facing messages for bad input cases.
- If AI/agents are used: implement one MCP integration and one A2A/ACP contract with tests.
- Add first test suite (unit + API/integration) and CI command.

Week 2 deliverables:

- runnable Scenario 1 demo path
- first dashboard page with at least 3 KPIs
- test report and failure screenshot/log

Week 3 — Scenario 2, reliability, and data acquisition

- Implement Scenario 2 from intake to intervention follow-up trace.
- Add one controlled failure case per scenario with safe recovery behavior.
- If AI/agents are used: validate ANP-style reliability rules (timeouts, retries, traceability).
- If scraping is used: implement source registry, polite crawler settings, and quality checks.
- Improve logs/monitoring views and role-based trace readability.

Week 3 deliverables:

- complete Scenario 2 demonstration
- failure injection evidence pack (screenshots + logs + expected behavior)
- scraping quality report (only if scraping track chosen)

Week 4 — Final evidence and presentation

- Finalize tests, metrics, and reproducibility documentation.
- Verify all claims map to concrete evidence (test output, logs, screenshots, reports).
- Prepare slides and live demo script (5–10 minutes, clear role-based story).
- Finalize risk register, governance notes, and limitations.
- Run full rehearsal, fix critical issues, and freeze final version.

Week 4 deliverables:

- final README with clean setup/run/test instructions
- final slides and demo package
- final metrics table and governance section

19 Submission Package

- **Moodle (mandatory):** upload the complete submission through the course **Moodle** assignment or activity indicated by the instructor. The due date is the **last Saturday of the semester** (see **Scheduling, Submission, and Individual Work**). Follow the Moodle instructions for file names and number of uploads.
- Slides (PDF or PPT): problem, architecture, demo flow, limits, next steps.
- Source code (repository or ZIP).
- README (install/run/test, sample accounts, dataset notes).
- **Datasets (when applicable):** if your project depends on data files (synthetic CSV/JSON, sample exports, fixtures, scraping snapshots, etc.), include them in the Moodle submission: either inside your ZIP archive or as separate uploaded files, and list them clearly in the README so evaluators can run your demo without hunting for data.
- Evidence: screenshots and/or short demo video.
- Short report section on ethics, data governance, and limitations.

20 Scheduling, Submission, and Individual Work

Exam session and validation

Project validation (oral defense, demonstration, and Q&A) takes place during the **main exam session** of this course. The exact date, time, and room list are **not specified in this handout**: they are set and communicated by the **administration of SESAME University**. Follow official university channels (timetable, portal announcements, and instructor notices).

Absence on validation day: if a student is **not present** on the scheduled validation day (defense, demonstration, and Q&A), the **maximum mark** that can be awarded for this exam project is **10 / 20**. Presence is required to claim the full **20 / 20** scale. Exceptional cases (documented force majeure) follow **only** written rules from SESAME administration or the instructor.

Submission

Mandatory: deadline, no extension, consequence of no submission

Deadline: submit the full package listed in Section 19 on the official course **Moodle** by **the last Saturday of the semester** (end of day as set in the Moodle activity).

No extra time: no extension and no late acceptance beyond that Moodle deadline will be granted.

No submission = 0: a student who **does not submit** the required work on Moodle by the deadline receives a mark of **0 (zero)** for this exam project.

Submit on **Moodle** only (official course space). Keep one reproducible archive (ZIP or repository link + any separate files Moodle allows) and ensure your README matches what you will run during validation.

Individual work (mandatory)

This exam project is **strictly individual**. Each student must design, implement, document, and defend **their own work alone**. Group work, shared codebases, or divided tasks among peers are **not permitted** unless the instructor issues an explicit written exception for a documented special case. Academic integrity rules apply: cite any external code or tutorials you reuse, and be prepared to explain every part of your submission.

21 Useful Links

A) Django engineering

- Django docs: <https://docs.djangoproject.com/en/stable/>
- Django security: <https://docs.djangoproject.com/en/stable/topics/security/>
- DRF docs: <https://www.django-rest-framework.org/>
- Graphene-Django docs: <https://docs.graphene-python.org/projects/django/en/latest/>
- Strawberry GraphQL docs: <https://strawberry.rocks/docs>

- gRPC docs: <https://grpc.io/docs/>
- OWASP cheat sheets: <https://cheatsheetseries.owasp.org/>

A.1) Agent and protocol ecosystem

- Model Context Protocol (MCP): <https://modelcontextprotocol.io/>
- Google A2A protocol: <https://google.github.io/A2A/>
- AGNTCY (ACP and ANP references): <https://agntcy.org/>

B) Data and social indicators

- UNICEF Data: <https://data.unicef.org/>
- WHO Data: <https://www.who.int/data>
- World Bank Tunisia data: <https://data.worldbank.org/country/tunisia>

C) Dataset building and validation

- Pandas IO guide: https://pandas.pydata.org/docs/user_guide/io.html
- Frictionless data specs: <https://specs.frictionlessdata.io/>
- Great Expectations docs: <https://docs.greatexpectations.io/docs/>

22 Student Quick Checklist

Mandatory checklist

Problem statement and stakeholders are explicit.

“Definition of done” conditions are all satisfied.

Scenario 1 implemented + failure evidence.

Scenario 2 implemented + failure evidence.

At least 2 advanced tracks completed with evidence.

If scraping is used: legality/robots/rate-limit/provenance documented.

Roles/permissions validated by tests.

Dashboard and reports operational.

README and setup commands verified.

Moodle submission before the last Saturday of the semester (no extension); otherwise mark **0**.

Full submission uploaded on Moodle; datasets (if any) included and listed in README.

Ethics and limitations clearly documented.

Individual work policy respected; ready for main exam-session validation (date/time per SESAME administration); **if absent on validation day, mark capped at 10/20**.

23 Final Note

The expected mindset is **responsible innovation**. Build something technically robust and socially useful. Your project should demonstrate how Django engineering can support youth well-being, educational continuity, healthier behaviors, and stronger support systems in Tunisia.